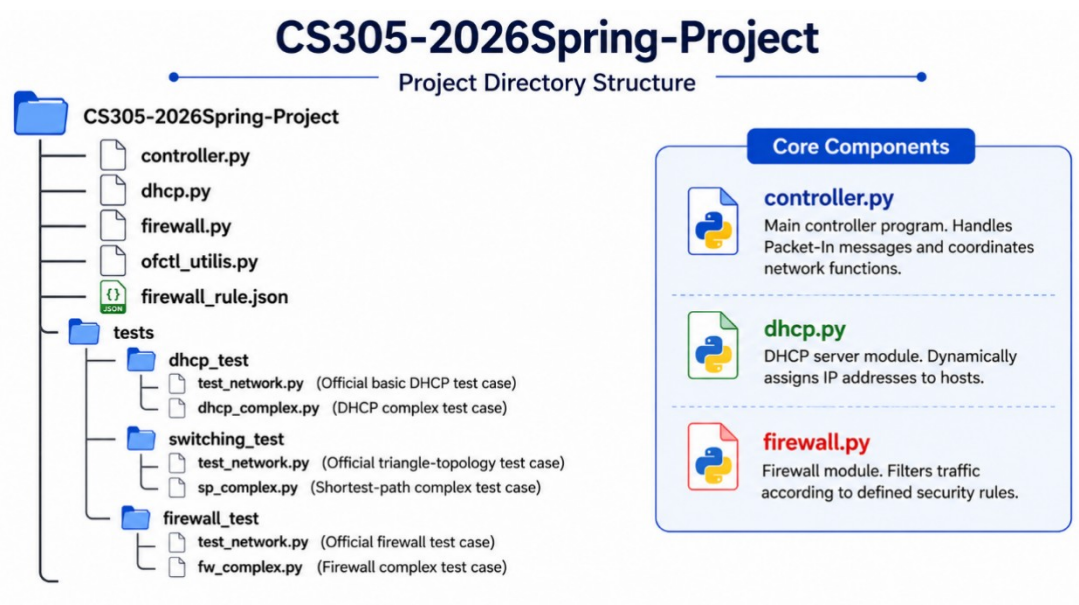


Project Architecture

This project is based on the Software Defined Networking (SDN) architecture and is implemented using Mininet, OpenFlow, and OS-Ken (Ryu). The system consists of a controller, switches, hosts, and testing modules. It provides three core functionalities: DHCP service, shortest-path switching, and firewall management.

Project Directory Structure



1 DHCP

1.1 DHCP Module Design

This project implements a DHCP server based on an SDN controller. When a host joins the network, the controller can identify DHCP protocol packets and automatically assign an available IP address to the host. Unlike a traditional DHCP server, the DHCP functionality in this project is centralized in the controller, while switches are only responsible for forwarding packets. This design demonstrates the separation of the

control plane and data plane in Software-Defined Networking (SDN).

To support dynamic address allocation, the system is preconfigured with DHCP server parameters, including the server MAC address, DNS server address, IP address pool, and subnet mask. The allocatable IP address range is set from 192.168.1.2 to 192.168.1.100, and the subnet mask is 255.255.255.0. All newly connected hosts obtain their IP addresses from this address pool.

1.2 IP Address Allocation Mechanism

The system manages address allocation by maintaining a mapping table between MAC addresses and IP addresses. When a DHCP request is received, the controller first checks whether the host has already been assigned an IP address. If a corresponding record exists, the previously assigned IP address is returned directly. Otherwise, the controller selects the next available IP address from the address pool and records the allocation in the mapping table.

This mechanism guarantees the uniqueness of IP assignments and prevents the same host from receiving different IP addresses when requesting repeatedly.

For address pool management, the system adopts a sequential allocation strategy. IP addresses are assigned in ascending order starting from the first address in the pool until the upper limit is reached. When the address pool is exhausted, the system generates an error message

indicating that no additional IP addresses are available for allocation.

1.3 DHCP Packet Construction and Processing

When a host first joins the network, it broadcasts a DHCP DISCOVER packet to locate a DHCP server. After receiving the packet, the controller extracts the host's MAC address and selects an available IP address from the address pool. The controller then constructs a DHCP OFFER packet containing the assigned IP address, subnet mask, DNS server information, and other network configuration parameters, and sends it back to the host through the switch.

After receiving the DHCP OFFER, the host broadcasts a DHCP REQUEST packet to confirm the selected configuration. Upon receiving this request, the controller retrieves the IP address previously assigned to the host and constructs a DHCP ACK packet as confirmation. The ACK packet includes the final assigned IP address, subnet mask, DNS server information, and lease time. Once the host receives the ACK packet, the DHCP configuration process is completed and the host is granted the right to use the assigned IP address.

The entire DHCP process is managed centrally by the controller. The controller first parses the DHCP message type field and performs different actions according to the packet type. When the message type is DISCOVER, the controller generates and sends a DHCP OFFER packet. When the message type is REQUEST, the controller generates and sends

a DHCP ACK packet. For unsupported DHCP message types or abnormal situations, the system records log information to facilitate debugging and maintenance.

1.4 Test Results

To verify the correctness of the DHCP module, two tests were conducted.

Test 1: Basic Functionality Test is a network topology consisting of two hosts and one switch was constructed. After startup, the hosts used dhclient to send DHCP requests. The controller successfully completed the DHCP DISCOVER, OFFER, REQUEST, and ACK process and correctly assigned IP addresses to both hosts.

Test 2: Multi-Host Test is a network topology consisting of four hosts and one switch was constructed. The four hosts sequentially sent DHCP requests. The controller successfully assigned a unique IP address to each host, and no duplicate address allocation occurred.

The test results demonstrate that the implemented DHCP module can correctly perform dynamic IP address allocation and operate reliably in a multi-host environment.

2 SDN Controller (controller.py)

2.1 Controller Module Design

This project implements a centralized SDN controller based on the os-ken framework. The file controller.py serves as the main entry point of the

system and coordinates three functional modules: DHCP address allocation (dhcp.py), shortest-path Layer-2 switching, and firewall-based access control (firewall.py).

Unlike traditional networks, in which control logic is distributed across switches and hosts, this design concentrates network control in a single controller process. Switches in Mininet only maintain OpenFlow flow tables and forward packets according to the rules installed by the controller. The controller is responsible for topology discovery, path computation, flow-table installation, DHCP processing, and coordination with the firewall module. This architecture clearly demonstrates the separation of the control plane and the data plane in Software-Defined Networking (SDN).

The controller is implemented as `ControllerApp`, a subclass of `app_manager.OSKenApp`, and supports OpenFlow 1.0. Internally, it maintains the following core data structures:

`switch_ofctl`: a mapping from switch DPID to OpenFlow helper objects, used to install flows on each switch;

`adjacency`: an adjacency list representing the current switch-level topology;

host_table: host location information indexed by MAC address;

ip_to_mac: a mapping from IPv4 address to MAC address, used for ARP handling;

firewall: an instance of the firewall module, used to install deny rules when switches connect.

When the controller starts, it waits for topology events from the os-ken topology discovery module. Whenever the network changes, the controller recomputes paths and reinstalls forwarding rules. This event-driven design allows the system to adapt to dynamic topology changes such as link failure, link recovery, switch stop/start, and host attachment.

For correct operation in Mininet, the controller should be launched with link observation enabled:

```
osken-manager --observe-links controller.py
```

This ensures that inter-switch link up/down events are detected and handled correctly.

2.2 Topology Discovery and Network Update Mechanism

The controller relies on the os-ken topology API to obtain global network

information. Switch-to-switch links are collected through `topo_api.get_all_link()`, and host information is obtained through `topo_api.get_all_host()`. Host discovery is typically triggered after hosts send ARP or gratuitous ARP packets during network initialization.

All topology-related event handlers invoke a unified function, `update_network()`, to refresh the global network state. This function performs four steps in sequence:

- rebuild the switch adjacency list via `_build_adjacency()`;
- retrieve all discovered hosts and register them through `_register_host()`;
- print the current topology and shortest switch paths for debugging and demonstration;
- install or refresh shortest-path forwarding rules on all switches.

The following topology events trigger `update_network()`:

This unified update pattern avoids duplicated logic across event handlers and guarantees that forwarding rules always reflect the latest topology. It is particularly important in dynamic test scenarios, such as `link s1 s2 down`, `link s4 s5 up`, `switch s3 stop/start`, or `ovs-ofctl mod-port s1 2 down`, where path recomputation must occur immediately after each topology change.

When a host is registered, the controller records its MAC address, attached switch DPID, ingress port number, and IPv4 address if available. If the host has a valid IPv4 address, the address is also inserted into `ip_to_mac` so that subsequent ARP requests can be answered correctly.

2.3 Shortest Path Switching Mechanism

The switching module implements centralized shortest-path forwarding based on global topology information. Its goal is to ensure that traffic between any two hosts follows a minimum-hop path across the switch fabric.

2.3.1 Topology Modeling

The controller converts the discovered switch topology into an undirected adjacency list. Each inter-switch link contributes one directed entry in adjacency:

```
adjacency[src_dpid][dst_dpid] = out_port
```

This representation is sufficient for unweighted shortest-path computation because all inter-switch links are treated as having equal cost.

2.3.2 Shortest Path Computation

For each destination host, the controller runs Dijkstra's algorithm from the destination switch `_dijkstra(dst_dpid)` and obtains the shortest hop distance from every reachable switch to that destination.

For each switch `sw` that can reach the destination, the controller selects the outgoing port toward the next hop using `_get_out_port_from_dist()`.

The rule is:

if `sw` is the destination switch, forward directly to the host port;

otherwise, choose the neighbor whose distance to the destination is smaller than the current switch's distance;

the corresponding port stored in `adjacency[sw][neighbor]` is used as the output port.

This produces a consistent next-hop forwarding decision on every switch along the shortest path.

2.3.3 Forwarding Rule Installation

After path computation, the controller installs OpenFlow forwarding entries on all relevant switches. Each rule matches IPv4 packets destined to a specific host MAC address:

match fields: Ethernet type IP, destination MAC (dl_dst);

action: output to the computed port;

priority: 100;

cookie: 0x305001.

These rules implement destination-MAC-based forwarding and allow switches to forward IP traffic without flooding. Because the controller recomputes paths after every topology update, forwarding automatically adapts when links fail or are restored.

For example, in the triangle topology used in the basic switching test, traffic from h1 to h2 initially follows:

h1 -> s1 -> s2 -> h2

If link s1-s2 fails, the controller recomputes the path and traffic is rerouted through the remaining triangle edge:

h1 -> s1 -> s3 -> s2 -> h2

In the complex ring topology, when direct ring segment s4-s5 is disabled, traffic between s4 and s5 is rerouted along the longer ring path. The controller log reflects this change, for example:

s4 to s5: s4 -> s3 -> s2 -> s1 -> s7 -> s6 -> s5, 6 edges

2.3.4 Path Verification and Logging

In addition to installing flows, the controller prints host-to-host shortest paths using breadth-first search over a combined host-switch graph. It also prints all-pairs shortest paths between switches. These logs are used to verify correctness during demonstration and testing.

Example controller output:

Topology (networkx): 14 nodes, 14 edges

s1 to s7: s1 -> s7, 1 edges

Path from h1(10.0.0.1) to h5(10.0.0.5): h1 -> s1 -> s7 -> s6 -> s5 -> h5,
distance=5

This output is especially useful in the complex test, where graders compare printed paths with the expected ring topology behavior after each dynamic operation.

2.4 ARP and Packet-In Processing

Not all packets are handled by preinstalled forwarding rules. The controller also processes selected packet-in messages to support address

resolution and host configuration.

When a switch sends a packet to the controller, `packet_in_handler()` parses the packet and dispatches it according to protocol type:

DHCP packets are forwarded to `DHCPServer.handle_dhcp()` in `dhcp.py`;

ARP packets are handled locally by `_handle_arp()`;

other packets are ignored by the controller and are expected to be handled by installed flow entries.

For ARP, the controller installs a high-priority packet-in rule on each switch:

match: Ethernet type ARP;

priority: 200.

When an ARP request arrives, `_handle_arp()` checks whether the requested destination IP exists in `ip_to_mac`. If the mapping is not yet available, the controller refreshes host information from the topology module and retries. If the IP is known, the controller constructs and sends an ARP reply through the requesting switch.

This mechanism allows hosts to resolve each other's MAC addresses even when IP forwarding rules are already installed. It is essential for

connectivity verification using pingall in switching tests.

If the controller log shows messages such as:

ARP request for unknown IP 10.0.0.1

it usually indicates that the host IP has not yet been registered correctly.

In practice, the controller should be restarted between major test cases to avoid stale host state left over from previous topologies.

2.5 Integration with DHCP and Firewall Modules

Although controller.py contains the switching and topology logic, it also acts as the integration layer for the other project modules.

DHCP Integration

When a DHCP packet is detected in packet_in_handler(), the controller delegates processing to the DHCP module. The controller itself does not assign IP addresses directly; instead, it identifies DHCP traffic and invokes the corresponding handler. This preserves modular design while keeping packet dispatch centralized.

Firewall Integration

When a switch connects in `handle_switch_add()`, the controller:

- creates an OpenFlow helper object for the switch;

- installs ARP packet-in rules;

- calls `self.firewall.install_rules(self.switch_ofctl)`;

- invokes `update_network()` to install forwarding rules.

Firewall deny rules are installed with higher priority than forwarding rules.

Therefore, even if a forwarding path exists, traffic matching a deny rule is dropped at the switch data plane before normal forwarding occurs.

The controller itself does not perform per-packet firewall inspection during runtime. Instead, the firewall module converts JSON rules into OpenFlow drop entries when switches connect. This matches the design described in the firewall module report: rule matching is performed in the data plane, while the controller is responsible for policy installation and topology-aware forwarding.

This priority assignment ensures correct interaction between security policy and routing behavior.

2.6 Topology Visualization and Logging

To satisfy the project demonstration requirements, the controller prints

the current network structure after each topology update using NetworkX.

The function `_print_topology_nx()` builds an undirected graph containing both hosts and switches, then prints:

total number of nodes;

total number of edges;

complete node list;

complete edge list.

The function `_print_switch_paths()` prints shortest paths and path lengths between every pair of switches. The function `_print_all_paths()` prints host-to-host shortest paths.

These logs serve two purposes:

they provide real-time evidence that the controller has detected a topology change;

they allow graders to compare controller output with the expected topology figure and path tables in the project report.

For the complex ring topology, the initial network contains 14 nodes and 14 edges. After operations such as link s4 s5 down, the controller should show updated switch paths while maintaining connectivity through

alternate ring segments.

2.7 Test Results

The Shortest Path Switching Algorithm is verified by `switching_test`.

A triangle topology with three hosts and three switches was constructed. After network initialization and gratuitous ARP, the controller installed shortest-path forwarding rules successfully.

Test procedure:

- start the controller with `--observe-links`;
- launch the switching test network;
- run `pingall` once to verify full connectivity;
- dynamically disable and restore links using Mininet CLI commands.

Expected results:

- initial `pingall`: 0% dropped;
- after disabling one triangle edge, hosts remain reachable through the alternate path;
- after disabling two edges incident to the same switch, the corresponding host becomes isolated;
- controller logs show updated switch paths after each topology change.

The test confirms that the controller can compute shortest paths correctly and adapt forwarding behavior in a topology containing loops.

2.8 Summary

The controller.py module is the core of the SDN project. It integrates topology discovery, shortest-path computation, OpenFlow forwarding installation, ARP handling, DHCP dispatch, and firewall coordination into one centralized control application.

Its main design features are:

event-driven topology maintenance through `update_network()`;

global shortest-path routing based on Dijkstra's algorithm;

destination-MAC-based flow installation on all switches;

modular integration with DHCP and firewall components;

detailed topology and path logging for demonstration and verification.

Experimental results show that the controller operates correctly in both basic and complex topologies, supports dynamic network changes through Mininet CLI operations, and provides reliable connectivity while cooperating with the firewall module for access control.

3 Firewall

3.1 Firewall Module Design

This project implements a software-defined firewall module based on the SDN controller, realizing flexible and fine-grained network traffic access control. Different from traditional hardware firewalls with fixed deployment and rigid rules, the SDN firewall decouples the control plane and data plane. The core rule judgment and traffic scheduling logic are centrally processed by the controller, while OpenFlow switches only execute the flow table forwarding and packet blocking instructions issued by the controller, which greatly improves the flexibility and scalability of network security management.

The firewall module adopts a separate configuration and test architecture following the project unified specifications. The core source files including `firewall.py` and `controller.py` remain unchanged. All security access rules are uniformly configured in the root directory `firewall_rule.json` file, realizing the separation of business logic and rule configuration. In terms of testing, basic function verification relies on the official original `test_network.py`, while complex scene robustness tests are independently implemented in the newly built `fw_complex.py` under the `tests/firewall_test` directory, which standardizes the module test structure and facilitates function verification and subsequent expansion.

The firewall supports customizable multi-type access control rules, covering common network traffic filtering scenarios such as specified

source IP, destination IP, source port, destination port and transport protocol. Users can add, modify and delete security rules by editing the JSON configuration file without modifying the module's core source code. Notably, rule changes only take effect after restarting the controller, as the system only loads the configuration file at startup and does not support real-time hot updates of security policies during runtime.

3.2 Firewall Traffic Filtering Mechanism

The firewall module adopts a rule matching first, default release filtering mechanism. The controller loads all predefined rules from `firewall_rule.json` at startup and caches them in an internal rule list. Once a switch connects to the controller, the controller actively converts all firewall rules into OpenFlow flow entries and installs them on all switches in advance. Different from per-packet controller detection, traffic filtering is completed directly at the switch data plane. If a packet matches the pre-installed deny flow entries, the switch discards it immediately; if no firewall rule is matched, the traffic proceeds to match the low-priority forwarding flow entries and is allowed to pass normally.

All firewall deny rules are configured with a unified high priority value (60000), which is significantly higher than the priority of normal forwarding flow entries (100), ensuring that interception rules take precedence over forwarding rules. The system traverses rules strictly in the array order defined in the JSON configuration file during flow table

installation, with no additional priority sorting logic for rules. Since all deny rules share the same priority, the final matching behavior for overlapping rules follows the inherent lookup mechanism of the OpenFlow switch. After deployment, the installed drop flow entries can permanently intercept all matching subsequent traffic, realizing persistent access control.

To adapt to complex network environments, the module supports multi-dimensional combined rule filtering. It can implement precise access control such as prohibiting specified single-host communication, blocking data transmission of designated protocol ports, and isolating communication between multiple network segments. This mechanism effectively avoids illegal access and abnormal traffic intrusion, and improves the overall security of the SDN network.

2.3 Firewall Rule Parsing and Packet Processing

The firewall module completes intelligent traffic filtering through three core steps: rule loading, packet parsing, and flow table delivery. Firstly, when the controller starts, it automatically reads and parses the `thefirewall_rule.json` configuration file, converts the JSON-formatted custom rules into program-readable rule objects, and completes the initialization of the firewall rule library.

Different from traditional controller-based traffic detection, this module does not parse or match packets dynamically through the

controller during network operation. The controller completes all rule parsing and flow table pre-deployment work at startup. All data plane traffic matching firewall rules is directly processed by the switch. The controller's packet-in handler only processes DHCP and ARP packets and does not participate in real-time firewall rule matching and traffic judgment.

The controller converts each parsed firewall rule into a standard OpenFlow drop flow entry and delivers it to all connected switches. The switch directly discards any packets that match the deny flow entries at the data plane without uploading packets to the controller. The current program implementation does not integrate interception log recording functions, and no log will be generated when the switch discards illegal traffic. For legal packets that do not match any deny rules, the packets will match the low-priority forwarding flow entries generated by the switching module to complete normal network transmission. The entire filtering process relies on pre-deployed flow tables for fast data plane processing, with high forwarding efficiency and stable performance.

3.4 Test Results

In accordance with the project's unified test specification, two levels of tests, including basic functional test and complex robustness test, are carried out for the firewall module to fully verify the accuracy and stability of the filtering function.

Test 1: Basic Functionality Test (Original test_network.py)

The basic test adopts the official standard single-switch multi-host topology. By configuring simple single firewall rules in `firewall_rule.json`, the basic interception function is verified. During the test, after starting the controller and the basic test network script, the host communication test is carried out. The results show that the firewall can accurately identify forbidden traffic matching the basic rules, successfully block illegal host communication, and normally release legal traffic that does not match the rules. The basic access control function works normally, which verifies the correctness of the core filtering logic of the module.

Test 2: Complex Robustness Test (Newly built fw_complex.py)

The complex test builds a multi-switch multi-host network topology in the newly created `fw_complex.py` test script, breaking through the limitation of the single-switch topology of the basic test. Meanwhile, multiple groups of multi-dimensional combined filtering rules are configured in `firewall_rule.json`, covering typical scenarios such as cross-switch host communication interception and designated TCP port access prohibition. The test adopts standard TCP-based HTTP service and communication commands, and verifies the interception effect of firewall rules on specified TCP port traffic, eliminating the incorrect setting of UDP protocol interception.

During the test, cross-switch communication requests between

different hosts are initiated to verify the firewall performance in complex network scenarios. The results show that the pre-installed firewall flow rules can accurately identify and intercept illegal cross-switch traffic specified by the rules, with no false interception or missed interception of legitimate traffic. For overlapping rule scenarios, the high-priority deny rules can effectively override normal forwarding rules to achieve precise access control. The test results fully prove that the firewall module has good robustness and scalability, and can effectively implement fine-grained security access control in multi-switch complex SDN network environments.

4 Bonus Features

In addition to the required DHCP, shortest-path switching, and firewall functions, two optional bonus features were implemented: configurable DHCP lease duration with server-side lease expiration and recycling, and pluggable shortest-path routing algorithms on the SDN controller. Both features reuse the existing Mininet test scripts and the centralized OS-Ken controller architecture.

4.1 Bonus Feature I: DHCP Lease Duration and Expiration Recycling

4.1.1 Design Overview

The basic DHCP module assigns IP addresses from a

configured pool and maintains a MAC-to-IP mapping table. The bonus extension adds RFC 2131 – style lease time signaling and server-side lease lifecycle management. Each binding is associated with an expiration timestamp. When a lease expires, the controller removes the MAC-to-IP entry so that the address can be managed consistently on the server side. Lease duration is configurable through the `lease_time` parameter in `dhcp.py` (in seconds). The value is advertised to hosts in DHCP OFFER and DHCP ACK messages using DHCP option 51 (Lease Time), encoded as a four-byte unsigned integer in network byte order.

Unlike a full production DHCP server, this implementation focuses on demonstrating lease configuration, renewal behavior, and expiration cleanup in the SDN lab environment. DHCP processing remains centralized in the controller; switches only forward packets.

4.1.2 Lease Management Mechanism

The DHCP server maintains two data structures: `allocated_ips`, which maps each client MAC address to its assigned IP address, and `lease_record`, which maps each MAC address to a wall-clock expiration time (`current_time + lease_time`).

When a DHCP DISCOVER message is received, the controller calls `_allocate_new_ip()`. If the client already has a non-expired binding, the same IP address is reused. Otherwise, a new address is

taken from the sequential pool (from 192.168.1.2 upward) and a new lease is recorded.

When a DHCP REQUEST is received, the controller sends a DHCP ACK. For the initial binding (SELECTING state, ciaddr = 0.0.0.0), the server refreshes lease_record. For renewal requests (RENEWING/REBINDING, non-zero ciaddr matching the bound address), the server still replies with ACK and option 51 but does not extend the server-side expiration time. This prevents the background dhclient renewal timer from indefinitely postponing expiration during bonus demonstrations.

The function clean_expired_lease() scans lease_record, deletes expired entries from both allocated_ips and lease_record, and logs [DHCP CLEAN] messages. The main controller application starts a periodic greenlet (_dhcp_lease_cleanup_loop) that invokes this function every five seconds (DHCP_LEASE_CLEANUP_INTERVAL). Optional log lines report the number of active leases and the time until the soonest expiration.

4.1.3 Interaction with the Controller

Expiration recycling is triggered only from controller.py; no additional changes to the Mininet test topology are required. On startup, the controller logs the configured lease_time, cleanup interval, and the approximate time until the first CLEAN message is

expected after an ACK.

4.1.4 Test Methodology and Results

Tests use the official tests/dhcp_test/test_network.py topology (two hosts, one switch). For bonus demonstration, lease_time is set to 10 seconds. The controller must be restarted after changing this value so that the running process loads the new configuration.

Test 1: Basic DHCP compatibility

After starting the controller and test script, h1 ifconfig and h2 ifconfig show addresses within 192.168.1.2 – 192.168.1.100. The required DHCP message exchange (DISCOVER, OFFER, REQUEST, ACK) completes successfully. This confirms that the lease extension does not break the basic DHCP grade requirement.

Test 2: Re-allocation after expiration

dhclient is run again on h1. The host receives a valid address from the pool. Server-side bindings have been cleared; the host interface may still display the old address until dhclient is run again, which is expected in this simplified model.

4.1.5 Summary

The bonus DHCP lease feature adds configurable lease duration in protocol messages, server-side lease tracking, renewal handling that does not block expiration demos, and timed recycling of expired bindings. It extends the base DHCP module without modifying

switching or firewall logic.

4.2 Bonus Feature II: Multiple Shortest-Path Routing Algorithms

4.2.1 Design Overview

The required shortest-path switching module computes minimum-hop paths over the switch topology and installs OpenFlow flow entries that match destination MAC addresses and forward to the correct output port. The bonus extension refactors this computation behind a pluggable routing interface so that the same forwarding and logging pipeline can use different classical shortest-path algorithms.

The topology is modeled as a directed graph: nodes are switch DPIDs, and edges come from `topo_api.get_all_link()`, stored in `adjacency[src_dp_id][dst_dp_id] = port_no`. All link weights are 1 (one hop per switch-to-switch link), consistent with the course requirement that path distance is the number of edges (e.g., $h1 \rightarrow s1 \rightarrow s2 \rightarrow h2$ has distance 3 including host links in the printed path).

4.2.2 Algorithm Selection and Unified Interface

The active algorithm is selected at controller startup via the environment variable `CS305_ROUTING`. Supported values are: `dijkstra` (default), `bellman-ford`, `bfs`, and `networkx`. Invalid values

fall back to dijkstra with a warning log. The chosen name is stored in `self._routing_algo` and printed at startup.

All algorithms implement the same interface `_shortest_distances_and_prev(src_dpid)`, returning:

`dist[v]`: shortest hop count from `src_dpid` to switch `v`

`prev[v]`: predecessor of `v` on a shortest path (for path reconstruction)

`_shortest_distances(dst_dpid)` is used for flow installation (reverse computation from the destination switch). Host-to-host path printing uses the same prev chain via `_switch_path_from_prev()` and `_compute_path()`, so logs and flow tables stay consistent with the selected algorithm.

4.2.3 Algorithm Descriptions

Dijkstra (heap-based): Uses a priority queue to relax neighbors with edge weight 1. Standard solution for non-negative weights; time complexity approximately $O((V+E) \log V)$.

BFS: Layer-by-layer expansion from the source. On an unweighted graph, the first visit to each node yields the shortest hop count. Equivalent to Dijkstra when every edge has weight 1, with simpler implementation.

Bellman-Ford: Repeatedly relaxes all edges for up to $|V| - 1$

rounds. Supports negative weights in general; on this project 's unit-weight topology it produces the same distances as BFS and Dijkstra.

NetworkX: Builds an undirected `nx.Graph` from the adjacency list and calls `single_source_shortest_path_length` and `single_source_shortest_path`. Predecessors are derived from the last hop of each shortest path. Demonstrates integration with a standard graph library.

4.2.4 Flow Table Installation (Unchanged Logic)

For each destination host, the controller runs the selected algorithm with `src_dpid = dst.port.dpid`. For every switch `sw` in the network, `_get_out_port_from_dist()` chooses the neighbor `n` with `dist[n] < dist[sw]` and installs a flow matching `dl_dst = host MAC` with action `output(port)`. If `sw` is the egress switch, traffic is sent to the host-facing port. This hop-by-hop greedy step along decreasing distance is unchanged from the original Dijkstra-only design; only the source of `dist` is swappable.

4.2.5 Why Results Coincide on the Triangle Topology

The official `tests/switching_test` uses a triangle of three switches and three hosts. With unit edge weights, every shortest-path algorithm listed above computes the same hop counts and the same switch sequences (e.g., `h1` to `h2`: `h1 → s1 → s2 → h2`, distance

3). The bonus therefore demonstrates correct multi-algorithm integration and stable connectivity rather than different paths on the same topology. Different paths would require weighted or asymmetric link costs, which are out of scope for the current adjacency model.

4.2.6 Test Methodology and Results

Tests use `tests/switching_test/test_network.py`. The controller must be restarted after changing `CS305_ROUTING`; Mininet should be cleaned with `sudo mn -c` between runs.

Test 1: Default Dijkstra

Without setting `CS305_ROUTING`, the controller logs Routing algorithm: dijkstra. After pingall, connectivity is 0% packet loss. The controller prints Host pair paths (routing=dijkstra): with expected shortest paths and distances.

Test 2: Bellman-Ford

Terminal 1: `export CS305_ROUTING=bellman-ford` then start `osken-manager --observe-links controller.py`. Repeat the switching test and pingall. Results: 0% loss; path strings and distances match Test 1; logs show routing=bellman-ford.

Test 3: BFS and NetworkX

Repeat with `CS305_ROUTING=bfs` and `CS305_ROUTING=networkx`. Connectivity and printed paths

remain the same on the triangle topology, confirming equivalence under unit weights.

Test 4: Topology change (optional)

Commands such as `link s1 s2 down` and `link s1 s2 up` trigger `update_network()`, which recomputes paths with the currently selected algorithm. Connectivity recovers after links are restored, showing that algorithm switching does not break dynamic topology handling.

4.2.7 Summary

The routing bonus adds a configurable shortest-path backend (Dijkstra, Bellman-Ford, BFS, NetworkX) behind a single `dist/prev` API, unified path logging, and environment-based selection via `CS305_ROUTING`. Forwarding behavior on the course topologies remains correct; the feature highlights algorithm pluggability and consistent integration with the SDN control plane.